

SafeTransition: exact rational certification of transition safety for sustainability assessment (venue-agnostic master, v1)

Amin Abaee

Independent Researcher — [ORCID: 0000-0002-0019-1842](https://orcid.org/0000-0002-0019-1842) — amin_abaee@ut.ac.ir

Abstract

SafeTransition, a Python library for certifying the transition safety of sustainability assessments, is presented. The library implements the assessment-operator framework: typed floor constraints evaluated on supplied tubes whose status is declared exact or certified conservative, scalarized aggregate operators over the positive weight cone, and backward recursions over state and belief spaces. Infeasibility is returned as a Farkas certificate, a nonnegative multiplier vector with vanishing weighted constraint sum and negative weighted bound; certificates serialize to canonical JSON for a standalone checker importing no library modules. All semantic assessment, recursion, threshold, and certificate computations run in exact rational arithmetic; floating point is confined to rendering and performance measurement. Twenty-four exact checks re-derive the companion deposit’s benchmark, and a scaling study over six parameterized families verifies closed-form answers to 256-variable eliminations and margins whose reduced denominators reach 323 bits. A single-file dashboard embeds the readings’ verification provenance. Distributed under the MIT licence.

Keywords: sustainability assessment; viability theory; transition safety; Farkas certificate; exact arithmetic; composite indicators; multi-criteria decision analysis.

1 Introduction

Composite indicators occupy a central place in sustainability monitoring. Aggregates reduce heterogeneous floor constraints on income, biomass, and social outcomes to a single weighted reading, and that reading is what dashboards, statutory reviews, and public communications display. The practice rests on a long modelling tradition, and its governance follows established frameworks for good model development and evaluation practice [Jakeman et al., 2006]. It also rests on a mathematical step that is easy to audit and easy to overlook: a weighted sum is certified positive at every audited snapshot, and the system behind the snapshots is declared safe.

The companion manuscripts show that this step can fail, and characterize exactly when it fails. In the assessment-operator framework of Abaee [2026a], a noncompensatory typed operator — which requires every floor to hold on the full trajectory, not at endpoints alone — can reject every plan that a scalarized aggregate operator licenses at a given weight; the separation is exhibited on an explicit rational witness datum and propagated through backward recursions. A resource-transition benchmark realizes the witness as a regulated fishery transition and verifies every quoted number in exact rational arithmetic. Under incomplete observation, the obstruction calculus of Abaee [2026b] complements viability assessment from the other side: it certifies that no observation-based policy is viable, through finitely checkable certificates — the common-action obstruction with an explicit Farkas multiplier vector, the observation-only safety-classification criterion, and a finite-horizon belief-space recursion that is sound for finite systems, with completeness of its label quotient

under a stated quotient condition. These results are mathematical; they describe what should be computed, and they certify verdicts conditional on the supplied rational datum — the framework is an exact audit layer for declared models, not a substitute for environmental modelling, statistical validation, or ecological judgement. They do not yet describe a tool that computes it.

This is the gap SafeTransition addresses. Reading a composite dashboard today, an analyst can recompute the aggregate with ordinary floating-point tooling, but has no standard way to re-derive the safety verdict behind it, to check that an infeasible plan menu is infeasible by certificate rather than by solver exit code, or to see which weight ratios license which plans. Numerical viability and control toolboxes compute kernels and invariant sets to high precision; multi-criteria decision analysis packages aggregate and rank; formal verification systems and satisfiability-modulo-theories solvers certify finite-state models and general constraint systems against their own specification semantics. What is missing in the intersection is a small, auditable layer in which the verdicts themselves — admissibility of a plan at a state, licensing at a weight, emptiness of a menu, emptiness of an epistemic kernel — are returned as exact rational objects that a third party can re-verify by hand or by machine, with the assessment semantics (floors, weights, licensed plan sets, partially observed beliefs) carried by the verdicts themselves rather than re-encoded.

This paper presents SafeTransition, a Python library that provides this layer. The related-software positioning of the paper was compiled against a preserved record of six verbatim web queries with dated result records (q1–q6, 2026), archived with the package inside the verification deposit; Section 5 reports what the searches covered and found. The library implements the five assessment operators and their accepted-state sets, the typed finite-graph backward recursion, the belief-space robust epistemic recursion, and the finitely checkable certificate family: exact rational Fourier–Motzkin elimination with provenance tracking, returning Farkas certificates and infeasibility margins; the finite common-action obstruction with a minimal conflicting subfamily; and the observation-only safety-classification criterion. Every semantic computation — assessment, recursion, threshold, and certificate arithmetic — runs in exact rational arithmetic; floats appear only in rendered graphics. The library re-derives, in twenty-four exact checks, the resource-transition benchmark of the verification deposit, and renders the resulting readings as a single-file dashboard. It depends on nothing beyond the Python standard library, which makes the exactness claim structural rather than procedural: there is no numerical backend whose rounds of approximation would need to be audited.

The paper is a software description. Section 2 states the mathematical basis as implemented, with proofs delegated to the companion manuscripts. Section 3 describes the software architecture and the exact-arithmetic discipline. Section 4 illustrates the library on the benchmark, on the dashboard readings, and on the partial-observation demonstrations. Section 5 positions the library relative to existing software, and Section 6 records availability and requirements. Section 7 concludes.

2 Mathematical basis

2.1 Transition data and assessment operators

A typed transition datum comprises a phase state $z = (q, x, s_1, \dots, s_n)$ with a phase flag q , fund coordinates x , and typed floors s_i normalized to zero; a finite action set; a finite set of disturbances with, for each action and disturbance, a tube $\text{Tube}(a, d)$ — the set of states visited over the review period, given here as piecewise-linear coordinate paths on breakpoints at which every visited extreme is attained; a transition-safe set S ; and a destination set G reached under an endpoint reset: the operators below evaluate tube constraints on the visited path, endpoint constraints at the

Tube status	Constraint set passes	Constraint set violated
EXACT	represented trajectory certified safe	represented trajectory certified unsafe
CONSERVATIVE	realization certified safe	inconclusive, unless the violating point is known reachable

Table 1: One-sided verdict semantics induced by the tube status. The asymmetry is the price of certified enclosure: a pass transfers through a conservative enclosure, a violation does not.

terminal pre-reset state y^- , and destination constraints at the post-reset successor $z^+ = R(y^-)$. Actions in the library declare offsets on the breakpoints, together with a finite action-indexed disturbance set. Each tube carries a status: **EXACT** (the exact visited set of its declared piecewise-linear paths) or **CONSERVATIVE** (a certified outer enclosure of an underlying nonlinear realization). The status determines the logical strength of verdicts: constraints passed on an exact tube certify the represented trajectory safe and a violation certifies it unsafe; on a conservative enclosure, passing certifies the realization safe, while a violation is inconclusive unless the violating point is known reachable. Table 1 summarizes.

Five operators are distinguished, following Abaee [2026a]; all share the same finite disturbance set per action, the same reset, and the same destination condition, and weights are strictly positive. For a state z , writing $\text{Tube}(a, d)$ for the visited set and $z^+(a, d)$ for the successor: the noncompensatory typed operator admits action a when $\text{Tube}(a, d) \subseteq S_{\text{typ}}$ — every floor plus the physical constraint — for every disturbance d and $z^+(a, d) \in G$; the scalarized aggregate operator replaces the coordinate-wise floors by $w \cdot s \geq 0$ at weight w , the physical constraint retained; the exact-tube physical operator drops the floors along the tube; the endpoint-only physical operator constrains only the terminal pre-reset state; and the typed-endpoint operator evaluates the floors at that pre-reset endpoint. Here $S_{\text{typ}} = \{y : s_i(y) \geq 0 \ \forall i, \text{Phys}(y)\}$ and $S_{\text{phys}} = \{y : \text{Phys}(y)\}$; S without subscript denotes the typed set in the recursions. Because floors with the physical constraint included imply their weighted shadow under positive weights, typed sets include physical ones, and tubes contain their endpoints, the chain

$$E_{\text{typ}}(z) \subseteq E_w(z) \subseteq E_{\text{tube,phys}}(z) \subseteq E_{\text{end}}(z)$$

holds for every strictly positive weight, each inclusion possibly strict: floors imply the nonnegative weighted sum when $w > 0$; deleting floors weakens the condition; a tube constraint implies the same constraint at the contained pre-reset endpoint; and the typed sets include the physical constraint by definition. The library verifies each inclusion exactly at any state on request (Section 3.3). The fifth operator continues the pattern on the typed side: $E_{\text{typ}}(z) \subseteq E_{\text{end,typ}}(z) \subseteq E_{\text{end}}(z)$, since typed floors constrain more than endpoints and more than physics alone. The accepted-state set $\mathcal{V}[E] = \{z : E(z) \neq \emptyset\}$ and the compensatory accepted set $\mathcal{V}_{\text{weak}} = \bigcap_w \mathcal{V}_w$ are derived from the operators. The compensatory set is the $\forall w \exists a$ condition — the licensed action may depend on the weight — and is in general strictly larger than the $\exists a \forall w$ uniform-action condition; this quantifier-order distinction is the companion separation in accepted-set form. The witnessed separation $\mathcal{V}_{\text{typ}} \subseteq \mathcal{V}_{\text{weak}}$, strict in general, is what a composite dashboard cannot see (Section 4.1).

2.2 Recursions

Two backward recursions are implemented. The typed recursion propagates viability backwards through an explicit state–action graph: with $\mathcal{W}_0$ the safe set, a state is $(k+1)$ -step viable when some action keeps every visited state inside the safe set and lands in the k -step viable set, quantified over the action’s finite disturbance set. On explicit graphs, under a unit-cost predicate model, the

recursion is polynomial in the numbers of states, state–action pairs, and horizon; bit complexity depends on rational coefficient growth and is reported separately by the scaling study (Section 3.4).

The belief-space recursion addresses partially observed systems, following the robust epistemic construction of Abaee [2026b]. States carry observation labels; beliefs are elements of the resulting quotient of the state space — sets of observation labels whose state fibres are re-expanded at every step. An action is admissible for a belief when no state in any of its labels’ fibres enters a violation en route under any disturbance, and the belief updates to the set of labels reachable under the action. The recursion returns the sets $\mathcal{W}_k$ of k -step viable beliefs, with $\mathcal{W}_0$ the safe labels. Soundness holds for all finite systems and for the natural policy class: a membership $\mathcal{B} \in \mathcal{W}_k$ certifies a belief-based (observation-history) policy viable for k steps. Completeness of the quotient — a belief outside $\mathcal{W}_N$ admitting no such policy — holds when the observation is a safety-and-action quotient (safe-set membership and action admissibility constant on fibres, successors label-determined); label injectivity on reachable states is the simplest sufficient condition. Absent the quotient condition, a non-membership is an abstraction artifact, not a certificate of policy nonexistence. A failure explainer accompanies a non-viable belief with its per-action counterexample: the exact reason each action fails — a fibre state entering a violation en route, or a post-belief that is itself not viable at the previous level — and each action is admissible in isolation in the state-wise, one-step, full-information sense. The failure object serializes as a certificate (system, horizon, per-action reason codes) that the independent checker re-derives by re-running the recursion from the serialized system alone. Post-observation recourse failures — beliefs in $\mathcal{W}_1 \setminus \mathcal{W}_2$ — are the signature output (Section 4.3).

2.3 Certificates

Infeasibility is certified, not merely reported. For a stacked rational system $Au \leq b$, the library runs Fourier–Motzkin elimination with provenance tracking: every derived row carries its multiplier vector over the original rows, and when a contradiction $0 \leq c$, $c < 0$ is derived, the derived multipliers $\lambda \geq 0$ with $\lambda^\top A = 0$ and $\lambda^\top b < 0$ are returned as a Farkas certificate, normalized to sum to one. The certificate is invariant to positive scaling of the multiplier vector; the normalized contradiction value $-\lambda^\top b > 0$ is fixed by the normalization but is not invariant under rescaling of the input rows, and is therefore reported as a normalized certificate contradiction value, not as an intrinsic distance to feasibility. The certificate is re-verified exactly before being returned; feasible systems return a bare verdict — feasibility is decided, not witnessed. The finite common-action obstruction fires when the safe-action sets of a mutually possible (co-possible under the assessed observation) family of states intersect emptily while each is nonempty — every state is individually viable under full information, yet no common action exists at this horizon; with recursively viable action sets the obstruction rules out a viable observation-based policy at that horizon. The library reports a minimum-cardinality conflicting subfamily, found by exhaustive search in nondecreasing size, so every strictly smaller subfamily is certified to intersect. The observation-only safety-classification criterion decides whether current safe-set membership can be classified exactly from the current observation alone: such a classification exists if and only if safe-set membership is constant on observation fibres, and a violating fibre is reported otherwise. The criterion is about this classification only: it does not rule out observation-history policies, belief-based control, conservative certifiers, or three-valued procedures. The certificate family comprises four serialized classes — Farkas infeasibility certificates, weight-partition certificates, benchmark certificates (which carry the tube-enclosure inequalities), and belief-failure certificates — each with a canonical serialization and an independent checker; two further findings are reported but deliberately not serialized (the minimum-cardinality conflicting subfamily and the violating fibre), their content being re-derivable

Module	Contents
<code>rational</code>	exact rational helpers (interpretation, rendering)
<code>datum</code>	typed transition data; the witness datum instance
<code>operators</code>	five operators, accepted-state sets, chain check
<code>recursion</code>	typed finite-graph recursion; belief-space recursion
<code>certificates</code>	Fourier–Motzkin with provenance; Farkas verification; common-action obstruction; safety-classification criterion
<code>indicators</code>	licensing thresholds, rescue threshold, blindness alarm
<code>dashboard</code>	single-file HTML rendering (inline SVG/CSS)
<code>benchmark</code>	twenty-four exact checks; verified schedule values; tube and benchmark certificates
<code>benchmarks/</code>	(directory, not a module) scaling study: six families, answers known by construction
<code>cli</code>	<code>verify</code> / <code>report</code> / <code>demo</code> / <code>certify</code>
<code>check_safe_transition_cert.py</code>	independent stdlib-only verifier for serialized certificates (repository root)

Table 2: Module organization of SafeTransition 1.3.0 (line counts by `wc -l` over the package tree: 1 698 library lines, 966 test lines, plus the 375-line independent checker and the 386-line scaling study).

by search and inspection rather than carried by a dedicated schema (Section 4.3, Table 6).

3 Software description

3.1 Architecture

The library is organized as nine modules, with the command-line entry point as an interface to them and the independent checker as a standalone repository-root script (Table 2). The `rational` module provides the exact rational helpers on which every other module rests, and the `datum` module defines the transition-data interface and ships one fully specified instance, the witness datum of the companion manuscript, with destination gain $e = (1/4, 1/4)$ and rescue cost $c = 1$. Custom data subclass the interface. The `operators`, `recursion`, and `certificates` modules implement Section 2; the `indicators` and `dashboard` modules produce the readings of Section 4.2; the `benchmark` module carries the twenty-four exact checks of Section 4.1. A command-line entry point exposes `verify`, `report`, `demo`, and `certify` — the last emits the serializable certificates of Section 4.3 for the independent checker. The `benchmarks/` directory (not a module) carries the scaling study of Section 3.4.

3.2 The exact-arithmetic discipline

Every assessment, recursion, and certificate is computed in exact rational arithmetic. Rationals enter either as integers, as decimal strings, or as explicit numerator–denominator pairs; no implicit conversion from binary floating point exists anywhere in the library, so an input’s exactness is structural, not a matter of reviewer discipline. Floats occur in exactly two places, both non-semantic: rendering geometry (SVG in the dashboard, matplotlib coordinates in figure scripts) and performance measurement in the scaling study. Checks never read floats. The distinction is the library’s design-centre claim: outputs are verifiable, not merely computed. A Farkas certificate, a licensing

threshold, or a benchmark check can be re-verified by any rational-arithmetic implementation — including pencil and paper — and the test suite exercises exactly this: every certificate returned by the eliminator is re-verified against its defining identities before being returned, and the benchmark checks re-derive each deposited value symbolically (for example $\sigma(16/5) = 1088/125$ for the sustained-yield quota of the Schaefer realization). The discipline is not merely hygienic: the suite includes an adversarial regression in which the exact rational predicate $49 \cdot (1/49) - 1$ evaluates to exactly zero, while under direct IEEE-754 binary64 evaluation in the tested expression order and with a strict zero threshold the same expression evaluates to -1.11×10^{-16} and would change the verdict. This is a regression case for one direct evaluation path, not a claim that floating-point tooling generally errs; SafeTransition rejects floating-point inputs structurally, so the failure mode is excluded by construction. Exactness is also stable under coefficient growth within the tested families: on a dyadic chain whose planted infeasibility gap is 2^{-t} , the certified margin retains a 323-bit reduced denominator at $t = 320$, long after binary floating point has rounded the gap away (Section 3.4).

3.3 Interfaces and complexity

The application programming interface mirrors the mathematics: `E(datum, z, mode, w)` returns the action set of an operator at a state; `check_chain` verifies the operator chain exactly; `belief_backward` returns the belief-viability sets $\mathcal{W}_k$ and `explain_belief_failure` the per-action counterexample for a belief outside them; `certify_polyhedron` returns a feasibility verdict with its certificate when infeasible (a bare verdict when feasible — feasibility is decided, not witnessed); `weight_partition` returns the complete licensing arrangement along the weight ratio as a serializable certificate, for the two-floor witness-family menu class whose per-plan trough conditions are affine in the ratio, over the admissible parameter family $0 < s_1, s_2 < \text{dip}$ (outside it a plan is licensed at every positive ratio or a threshold denominator is nonpositive, and the routine raises rather than misinterprets); `compute` returns the dashboard readings at a state, weight, and read plan. The typed recursion on explicit graphs is polynomial in graph size and horizon; Fourier–Motzkin elimination can generate exponentially many intermediate inequalities in the worst case. Exact rational linear feasibility is not inherently exponential — polynomial-time algorithms exist in the bit model — but the library uses Fourier–Motzkin elimination because the targeted assessment systems are small and row-level provenance is straightforward to retain (a handful of constraints; the witness instances eliminate in milliseconds). Belief enumeration is bounded — the bound is a parameter (default 4096 reachable beliefs) whose exhaustion raises an exception carrying no partial results; the bound affects exploration completeness, never the soundness of returned memberships.

3.4 Scaling of the exact pipeline

The precision–performance trade-off is measured, not asserted. A scaling study sweeps six parameterized instance families whose answers are known by construction and asserted at every size: stacked Fourier–Motzkin chains with a planted infeasibility gap (chain length up to $k = 256$ variables), dyadic coefficient growth (exact margins certified at denominators 2^t up to $t = 320$), a dense random stress family with planted answers (reporting where classical row growth becomes impractical), typed recursions on layered graphs (up to 21 000 states at horizon 20), cloned action menus (up to 512 actions through all five operators), and belief systems whose reachable belief count is exactly 2^m (largest completed $m = 12$ with all 4096 beliefs; the overflow test $m = 13$ raises with no partial results). The study records wall-clock runtime, peak traced memory, eliminator row statistics

Family	Known answer (asserted)	Measurement at the stated instance
FM chain, largest run $k = 256$	margin $(1/5 - 255/4096)/257$	1.59 s eliminator; 256 rows; 335 kB certificate; checker 0.29 s
Dyadic growth, largest run $t = 320$	margin $2^{-320}/7$	certified exactly; margin 1 numerator / 323 denominator bits
Dense stress, 4 variables (12 rows), infeasible	Farkas verified	7 505 intermediate rows; 2.2 s
Dense stress, 4 variables, feasible	feasible	impractical: 60 s budget exceeded (recorded as the envelope)
Typed recursion, 21 000 states	all states viable	2.1 s; linear in states
Action menu, 512 actions	all admissible, five modes	2.5 s
Beliefs, largest completed $m = 12$	2^m reachable beliefs	4 096 enumerated; overflow test $m = 13$ raises, no partial results

Table 3: Scaling of the exact pipeline (full sweep, single run per instance, no warm-up, wall-clock; Intel Xeon @ 2.60 GHz, Linux x86-64, CPython 3.13; checker times include subprocess startup; memory via `tracemalloc`, which adds overhead to traced phases; the committed run is regenerated by `benchmarks/scaling_study.py`). Every instance’s answer is known by construction and asserted.

(rows generated, peak working rows), certificate sizes with maximal numerator/denominator bit lengths reported separately, and independent-checker runtime (Table 3). Within these structured planted-answer families, exact rational certification remains stable as coefficient size and instance dimension grow: the chain family generates k rows and terminates at the planted contradiction (the certificate JSON reaches 335 kB at $k = 256$, with 13-bit coefficients, re-verified by the independent checker in 0.29 s), the typed recursion is linear in the state count (21 000 states in 2.1 s), belief enumeration is exponential in m as the reachable set itself is, and the bound fires where designed. For the chain family the margin has the closed form $(1/5 - (k - 1)/4096)/(k + 1)$, verified at every k ; for the dyadic family (chain of $L = 5$ intermediate links) the margin is exactly $2^{-320}/7$, whose reduced denominator $7 \cdot 2^{320}$ has 323 bits. Within the tested range, coefficient growth had a larger observed effect on certificate size than on wall-clock time. The planted families measure correctness and exact-arithmetic stability under controlled growth; the dense stress family reports the classical row growth without deduplication (975 intermediate rows at 3 variables; 7 505 rows at 4 variables in 2.2 s for the infeasible case) and the point at which elimination becomes impractical — the 60 s budget fires at 4 dense feasible variables — so the dense-system envelope of the eliminator is narrow, as the complexity statement above says.

4 Illustrative examples

4.1 The resource-transition benchmark

The benchmark realizes the witness datum as a regulated fishery transition: stock biomass with Schaefer surplus $\sigma(B) = rB(1 - B/K)$ ($r = 4$, $K = 10$, $B_{\text{lim}} = 2$), a pulse-plus-closed-season quota plan, a mid-season heatwave strike, and a staged rebuild financed by a fund buy-back. Three layers are distinguished: the exactness claims are relative to the supplied rational datum; the enclosure claims connect that datum to the Schaefer realization; and the ecological adequacy of the Schaefer model itself is outside the scope of the certification layer. Running `safetransition verify` executes twenty-four exact checks that re-derive the deposited verification values: the witness datum and tube tables; quota admissibility on both disturbance branches ($0 \leq H^* \leq 13$,

Artifact	Result
Benchmark exact checks (<code>safetransition verify</code>)	24/24 pass
Unit and integration test suite	59/59 pass
Runtime of checks + tests (Xeon @ 2.60 GHz)	under 5 s
Farkas certificate, stacked menu window	$\lambda = (1/2, 1/2)$, margin 1/10, re-verified
Independent checker over emitted certificates	3/3 verified; tampered certificates rejected
Scaling study, FM chain $k = 256$	closed-form margin verified; checker 0.29 s
Dashboard provenance	deterministic render; certificate SHA-256 embedded
Figure provenance	regenerated values equal deposited values (tested)
Adversarial exact/float instance ($n = 49$)	exact licensed; float would unlicense
Belief recursion, recourse-failure system	root belief in $\mathcal{W}_1 \setminus \mathcal{W}_2$
Source / test lines (plus checker, study)	1698 / 966 (+375, +386)

Table 4: Verification artifacts shipped with SafeTransition 1.3.0. Every check and certificate is evaluated in exact rational arithmetic.

peak $1463/125$); tube-status certification — the declared piecewise-linear plan tubes are exact relative to their declaration, while the Schaefer realization’s tubes are certified conservative outer enclosures: on the adverse recovery leg biomass lies in $[6/5, 16/5] \subset (0, 5) = (0, K/2)$, where $\sigma'(B) = 4 - 4B/5 > 0$, so σ is increasing on the interval and its minimum is the left endpoint value $\sigma(6/5) = 528/125 = 4.224 \geq 4$, which certifies the enclosure on the recovery legs; the index blindness of the composite reading at $w = (1, 1)$, whose minimum stays at $2/5 > 0$ along the licensed plan while the ecological floor reaches $-4/5 < 0$ mid-transition; the licensing thresholds $\rho_1 = 2/3$ and $\rho_2 = 3/2$, cross-checked against the thresholds derived directly from the tube geometry; and the rescue threshold $\kappa^* = 1 - x$ on the non-typed-viable region, truncated at zero ($\kappa^* = \max(0, 1 - x)$) once the fund covers the buy-back, with shortfall $1/2$ at the false-positive witness and full financing at the rescue witness $x = 3/2$. All twenty-four checks pass in under a second; Table 4 collects the verification artifacts. Because the realization tubes are conservative, the benchmark’s one-sided reading is: plans whose declared tubes pass are certified safe, and the transfer to the nonlinear realization holds by the enclosure; a plan rejected on an enclosure violation would be not-certified-safe rather than certifiably unsafe, unless the violating point is reachable — the benchmark’s separation rests on the declared (exact) plan tubes, so both directions are conclusive there.

Figure 1 shows the benchmark panels regenerated from the same verified values. Panel (a) is the separation a composite index cannot see: the weighted reading stays certified over the full review period while the floor it aggregates is driven below zero mid-transition. Panel (b) is the plan menu as management schedules: the pulse-plus-closed-season quota, the sustained-yield alternative at $\sigma(16/5) = 1088/125$, and the staged rebuild below yield while the fund finances the buy-back.

4.2 Dashboard readings

The `compute` interface returns, at a state and weight, the per-plan floor minima under each plan’s characteristic disturbance — an illustrative branch, displayed as such; the robust operator verdicts quantify over all disturbances — the composite index along the read plan, the licensing verdicts at the weight ratio $r = w_2/w_1$, the rescue threshold, and the index-blindness alarm, which fires exactly when the composite minimum stays nonnegative (the aggregate-licensed condition, boundary included) while a floor minimum is negative. In this subsection the pairs $(1/2, 1/2)$ and $(3/2, 0)$ denote $(x, \kappa^*(x))$ — fund level and shortfall — not the phase-state coordinates of Section 2.1, and the phase flag q is suppressed in the displayed triples, being fixed at the review-phase value throughout. At the witness state $(x, s_1, s_2) = (1/2, 6/5, 6/5)$ with $w = (1, 1)$, the alarm fires: the readings repro-

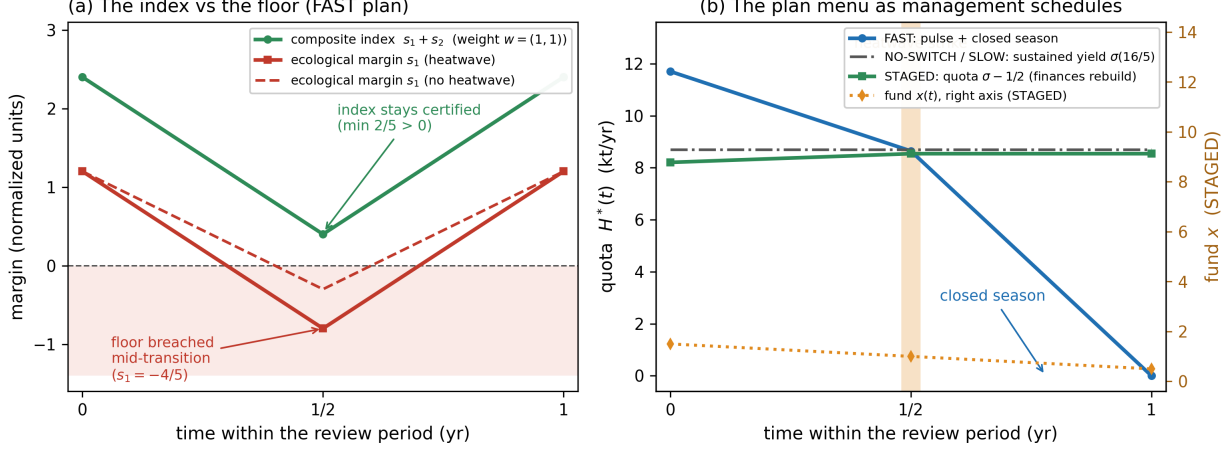


Figure 1: The resource-transition benchmark, regenerated from the library’s exact checks. (a) The composite index at $w = (1, 1)$ stays certified ($\min 2/5 > 0$) along the licensed plan while the ecological margin s_1 is breached mid-transition ($-4/5$ adverse, $-3/10$ benign): the separation the typed operator exposes. (b) The plan menu as management schedules: FAST (pulse 1463/125 plus closed season), SLOW/NO-SWITCH (sustained yield 1088/125), and STAGED (quota $\sigma - 1/2$, rebuild to 69/20), with the fund schedule $x(t)$ under STAGED on the right axis. Regenerated by `make_benchmark_v45.py` from the library’s verified schedule values, which equal the deposited values (equality tested).

duce the benchmark separation as a dashboard statement (Fig. 3). The `dashboard` module renders the readings, the benchmark status, and any certificates as a single HTML file with inline SVG and CSS and no external resources, so the file can be attached to a review, emailed, or archived with a statutory record and remains readable offline. The dashboard is an audit-delivery mechanism rather than a novelty claim, and it is bound to the certificate schema: a verification-provenance table embeds the library version, the independent checker’s version and re-run command, the datum identifier, and the SHA-256 hash of every certificate’s exact serialized JSON, expandable to the serialization itself. Rendering is deterministic — a pure function of the inputs, with no timestamps — so a re-render from the same inputs is byte-identical and the embedded hashes are checkable against the emitted certificate files. From the reviewer’s perspective this means the reading and its verification arrive together: thresholds, alarms, certificates, hashes, and the re-run command in one file. The hashes establish integrity and identity of the serialized inputs, not their authenticity or external truth: a producer could embed mutually consistent false data, and the dashboard verifies consistency with the serialized datum, not the world.

Figure 2 shows the resulting trusted computing base as a certificate chain from the supplied datum to the dashboard provenance block.

4.3 Partial observation and certificates

The bundled demonstrations exercise the certificate family on the companion manuscript’s instances. The three-state recourse failure (Table 5) runs the belief-space recursion on a system whose root belief is one-step viable and exits at the second step under every observation-based policy: the root belief lies in $\mathcal{W}_1 \setminus \mathcal{W}_2$, witnessed only by the recursion. The observation-only safety-classification criterion reports the merged fibre $\{x_1, x_2\}$ on which safe-set membership varies,

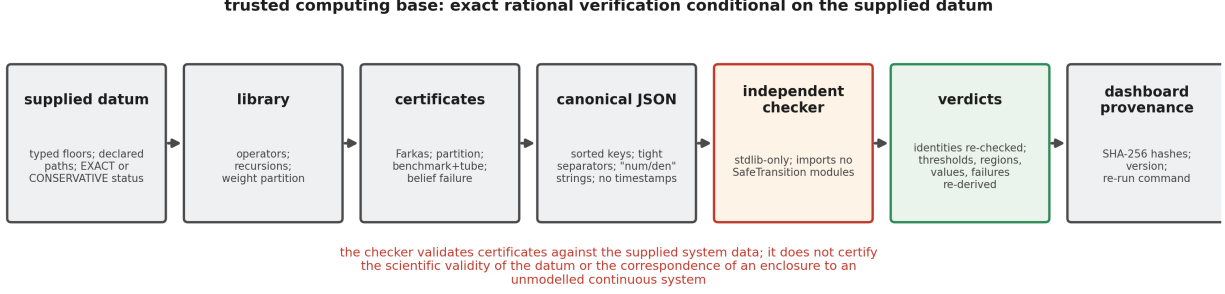


Figure 2: The certificate chain: the trusted computing base of an assessment verdict. Exact rational verification is conditional on the supplied datum; the checker validates certificates against the supplied system data and does not certify the datum’s scientific validity.

State	Observation	Actions	Successors per action	Safe
x_1	y_{12}	a, b	$a \mapsto x_1; b \mapsto x_4$	yes
x_2	y_{12}	a, b	$a \mapsto x_4; b \mapsto x_2$	yes
x_4	y_4	a, b	both $\mapsto x_3$	yes
x_3	y_3	a, b	both $\mapsto x_3$	no

Table 5: The recourse-failure system, fully specified. Root belief $\{y_{12}\}$ lies in $\mathcal{W}_1$ (both actions keep every fibre state safe at the first step), but the post-belief under either action is $\{y_{12}, y_4\} \notin \mathcal{W}_1$, since x_4 inevitably reaches the violating x_3 ; hence $\{y_{12}\} \in \mathcal{W}_1 \setminus \mathcal{W}_2$.

ruling out exact classification of current safe-set membership from the current observation alone; it does not address history-dependent procedures. The stacked menu window $u \leq 2/5$, $-u \leq -3/5$ returns the Farkas certificate $\lambda = (1/2, 1/2)$ with infeasibility margin exactly $1/10$, re-verified against $\lambda^\top A = 0$ and $\lambda^\top b < 0$ before being returned. The mapping to the obstruction calculus is row-explicit: row $[1]$ $u \leq 2/5$ is the first state’s menu cap, row $[-1]$ $u \leq -3/5$ is the second state’s floor requirement, the certificate’s multiplier vector weights the states’ restrictions equally, and the empty common menu the obstruction reports is exactly the infeasibility these two rows certify — one Farkas object, addressable row by row to the states whose compatibility it denies. A deposited double-integrator identity (weights $(3/8, 5/16, 5/16)$ with $\sum_j \lambda_j n_j = 0$) is checked exactly; its system, normals, and role as a regression anchor are tabulated in the Supplementary Material (S11). On the recourse-failure system, `explain_belief_failure` returns the counterexample object: at the root belief both actions are admissible in isolation in the state-wise, one-step, full-information sense, and each fails because its post-belief $\{y_{12}, y_4\}$ is not one-step viable — x_4 ’s only successors enter the violating x_3 — the machine-checkable witness behind the $\mathcal{W}_1 \setminus \mathcal{W}_2$ verdict. The object serializes as a `belief_failure` certificate — system, horizon, and per-action reason codes — which the independent checker verifies by re-running the recursion from the serialized system alone through a stdlib-only reimplementation; a tampered reason or post-belief is rejected.

The weight-space analysis is complete rather than pointwise. The `weight_partition` interface returns the full arrangement of licensed plan sets along the ratio $r = w_2/w_1$ as a serializable certificate. For the benchmark floors the arrangement is the three regions the readings display: SLOW-only on $[0, 2/3]$, both plans licensed on $[2/3, 3/2]$, FAST-only on $(3/2, \infty)$ (weights with a zero component are handled by projective closure: a plan is licensed at $r = 0$ or $r = \infty$ exactly when its floor conditions degenerate so at the closure). Across the admissible rational family —

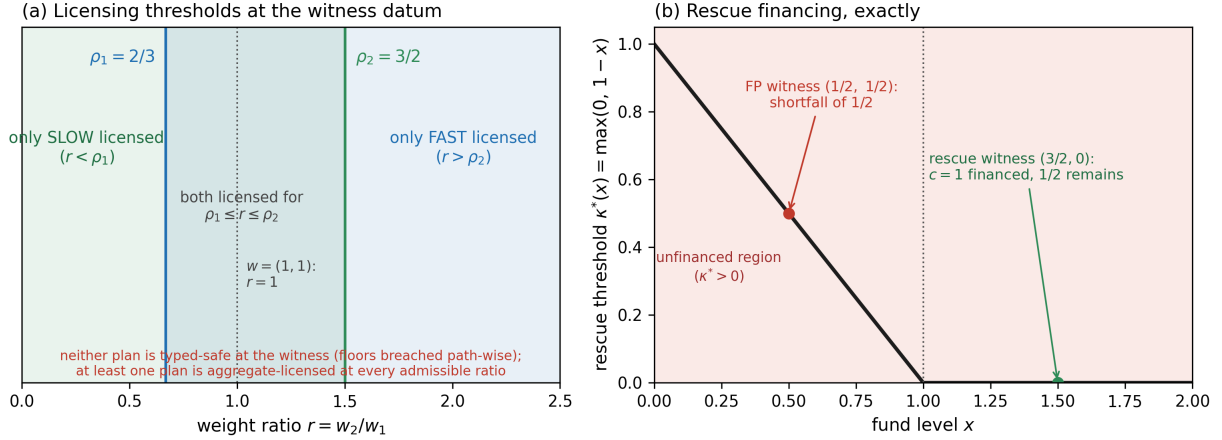


Figure 3: Dashboard readings at the witness datum, exact. (a) Per-weight licensing: the aggregate operator licenses FAST at weight ratios $r \geq \rho_1 = 2/3$ and SLOW at $r \leq \rho_2 = 3/2$; the standard equal-weight reading $r = 1$ sits inside the band where both are licensed, and at no admissible ratio is a typed-safe plan licensed, although at least one plan is aggregate-licensed at every admissible ratio. (b) Rescue financing: $\kappa^*(x) = \max(0, 1 - x)$; the false-positive witness $(1/2, 1/2)$ carries a shortfall of $1/2$, the rescue witness $(3/2, 0)$ finances the buy-back with $1/2$ remaining.

floor pairs (s_1, s_2) and dip depths with $0 < s_1, s_2 < \text{dip}$, outside which `weight_partition` raises (a plan licensed at every positive ratio, or nonpositive threshold denominators) — the arrangement has three regimes: for $\text{dip} < s_1 + s_2$ (the benchmark’s) the FAST threshold $(\text{dip} - s_1)/s_2$ precedes the SLOW threshold $s_1/(\text{dip} - s_2)$; for $\text{dip} > s_1 + s_2$ the thresholds swap and an unlicensed gap opens around $r = 1$; at equality the middle region degenerates to the single point where both plans bind simultaneously. The independent checker re-derives, from the certificate alone, the thresholds, region coverage and inclusivity, the licensed set at a sampled ratio in each region, and the binding witness constraint at each boundary.

5 Relation to existing software

SafeTransition complements three established families of tools, and borrows from each. Numerical viability kernels and reachability tools compute set-valued safety objects to high floating-point precision with impressive scalability; they answer *how large is the safe set*, where SafeTransition answers *is this verdict verifiable*, for the small, structured, rational instances that assessment menus produce. Modular environmental simulators with public source code — Raven [Craig et al., 2020] for watershed modelling, FloPy [Bakker et al., 2016] as a MODFLOW model-builder — are the nearest workflow neighbours: they consume declared scenario data and produce scenario outputs, and a viability or reachability layer built on top of them is where set-valued safety computation would live. Multi-criteria and sensitivity analysis packages — the SAFE toolbox [Pianosi et al., 2015], SALib [Herman and Usher, 2017], SPOTPY [Houska et al., 2015] — share the library’s workflow-oriented design: model-independent interfaces, worked examples, and benchmark functions; SPOTPY’s demonstration that one package can carry an entire family of methods with a minimal amount of glue code is the presentation standard SafeTransition aims at. Environmental simulators with executable regression suites — Pywr [Tomlinson et al., 2020] and airGR [Coron et al., 2017] — inform the availability discipline: permissive licensing, distribution through package indexes, and

Certificate class	Emitted	Independently checked	Checker re-derivation
Farkas infeasibility	yes	yes	multiplier identities ($\lambda \geq 0$, $\lambda^\top A = 0$, $\lambda^\top b < 0$)
Weight partition	yes	yes	thresholds, region tiling and inclusivity, licensed sets at sampled ratios, boundary witnesses
Benchmark + tube enclosure	yes	yes	eleven deposited values; monotone-interval enclosure inequalities
Belief failure	yes	yes	recursion re-run from the serialized system; per-action reasons
Feasibility	bare verdict (no witness, by design)	—	—
Obstruction subfamily; violating fibre	reported, not serialized	—	re-derivable by exhaustive search / fibre inspection

Table 6: Certificate support matrix, stated honestly per class: four serialized classes are emitted and independently checked; feasibility is decided without a witness (Section 2.3); the two unserialized findings are re-derivable but carry no dedicated schema.

a regression suite any user can re-run. None of the environmental families just listed returns certificates or computes in exact arithmetic (a claim about the packages surveyed in the archived novelty search, not about all environmental software). General-purpose provers do both in their own domains. The z3 satisfiability-modulo-theories solver [de Moura and Bjørner, 2008] decides linear real arithmetic exactly over the rationals and emits checkable unsatisfiability cores; its default cores, however, are trace proofs of its own inference rules, not self-contained algebraic witnesses for a third party who does not run z3, whereas a Farkas certificate is verifiable by thirty lines of school arithmetic. The Fourier–Motzkin projection library pyfine [Caron, 2022] performs exact symbolic elimination over SymPy rationals with Imbert acceleration. Exact polyhedral computation is itself an established discipline: reverse-search vertex enumeration as implemented in `lrslib` [Avis, 2000] and Hilbert-basis computation as implemented in `Normaliz` [Bruns and Ichim, 2010] compute polyhedral objects exactly and emit partially self-verifying outputs; the objects they certify are geometry, not assessment semantics. Probabilistic model checking in PRISM [Kwiatkowska et al., 2011] verifies finite-state probabilistic models against temporal-logic specifications and returns counterexamples and witnesses, at floating-point scale for numerical queries. These tools are not certificate-free; the point is scope. What none of them ships is assessment semantics: floors, weight-licensed plan menus, reachability tubes for transition data, and epistemic viability sets are objects the analyst must encode afresh, and the encoding, not the prover, then carries the burden of exactness. Table 7 summarizes the comparison; the six verbatim search queries behind it (q1–q6, with dates, trimmed result records, and the mapping to the claims they support) are archived in the package’s `novelty_searches/` directory inside the verification deposit [Abae, 2026], the figshare item 10.6084/m9.figshare.33764023 whose master copy doubles as the software’s verification deposit. The comparison is functional, not competitive: the families operate at scales and purposes for which floating-point computation is the right engineering choice, and the exact layer becomes decisive precisely where verdicts enter statutory or adversarial settings and must be re-derivable by third parties.

Property		SafeTransition	z3	PRISM	pyfme
Exact arithmetic	rational	throughout	yes (linear real arithmetic)	no (floating point)	yes (SymPy rationals)
Verdicts by third parties	checkable	yes; Farkas certificates, independent checker	unsat proofs	cores, counterexample traces	no provenance
Assessment semantics (floors, licensing)	tube se-	built in	re-encoding required	re-probabilistic models	polyhedra only
Weighted-aggregation analysis	anal-	licensing per ratio; complete weight partition	encoding-level	—	—
Belief-space recursion	viabil-	epistemic sets $\mathcal{W}_k$ with failure witnesses	encoding-level	POMDP algorithms (numerical value iteration; certificates are policy-dependent)	—

Table 7: Functional comparison. The environmental toolsets discussed in the text (Raven, FloPy, Pywr, airGR, SAFE, SALib, SPOTPY) are numerical throughout and return no certificates; they are omitted from the columns for space.

6 Availability and requirements

The test suite (59 tests) covers the benchmark, the operator chain and witnessed separation on the witness datum, the certificate family on the deposited instances, the certificate protocol (serialization, independent checking, tamper rejection, import hygiene, the adversarial exact/float instance) — all executed on CPython 3.13, the only interpreter on which the suite has been exercised — the scaling study’s closed-form answers on small instances, figure provenance (regenerated values equal deposited values), dashboard provenance determinism, the belief-space recursion on the recourse-failure system, and the dashboard’s self-containment. A property-based battery adds seeded randomized chain inclusion and Farkas validity (60 instances each, the latter cross-checked against an independent exact vertex-enumeration oracle), belief-recursion monotonicity, partition coverage sampling including degenerate regimes, cross-process hash stability of the canonical serialization, mutation testing of the certificate protocol, and degenerate systems (zero rows, duplicate rows, coincident thresholds, empty action menus, uneven action menus across an observation fibre); the whole suite runs in under five seconds under the standard library’s `unittest`. The suite is executed locally and archived with its outputs; no continuous integration service is claimed. The dashboard example regenerates the single-file HTML from the library. Section-level derivations, the complete enumeration of the twenty-four benchmark checks, a worked certificate derivation, the package artifact tree with its reproduction procedure, and the related-software search records supporting Section 5 are provided in the Supplementary Material (S1–S11).

Name	SafeTransition
Developer and contact	Amin Abaee; amin_abaee@ut.ac.ir
Year first available	2026
Hardware required	none beyond a standard computer (all timings reported here: Intel Xeon @ 2.60 GHz, Linux x86-64, CPython 3.13)
Software required	Python ≥ 3.9 ; no packages beyond the standard library (matplotlib and Pillow only for regenerating figures; publicly maintained third-party distributions are listed with their pinned versions in the verification deposit)
Program language	Python
Program size	1 698 library lines, 966 test lines, 375-line independent checker, 386-line scaling study (<code>wc -l</code>)
Availability and cost	open source under the MIT licence; source, tests, and examples archived in the verification deposit [Abaee, 2026] (figshare, 10.6084/m9.figshare.33764023) and mirrored at https://github.com/MIKEAA2020/general-sustainability ; the release archive ships a SHA-256 integrity manifest; no cost

Table 8: Software availability summary.

7 Conclusions

7.1 Limitations

Four limitations bound the tool’s use. First, exactness is relative to the supplied datum: the certification layer validates verdicts against declared rational data, not against the modelled system, and enclosure claims transfer only as far as the certified enclosure (Section 4.1). Second, the belief recursion’s completeness is conditional on the safety-and-action quotient; outside it, non-membership is an abstraction artifact (Section 2.2). Third, the eliminator’s dense-system envelope is narrow — the stress family becomes impractical already at four dense feasible variables — so the certificate family targets the small stacked systems that assessment menus produce (Section 3.4). Fourth, the weight partition covers its stated two-floor menu family; menus outside the family require new derivation rather than reparameterization (Section 4.3).

SafeTransition turns the assessment-operator framework and the obstruction calculus of the companion manuscripts into auditable software: typed operators evaluated on tubes whose status is declared exact or certified conservative, recursions over state and belief space, and infeasibility returned as Farkas certificates with exact margins, serialized and re-verified by an independent checker sharing no code with the library. The library’s scope is deliberate: it certifies verdicts on small, explicit, rational instances under exact or certified conservative tubes, and it makes no predictive claim — the theorem-level guarantees are those of the companion manuscripts, and on continuous or belief-continuous systems the graph construction itself is the modelling step. Within that scope, the tool addresses a practical gap: readings of the kind composite dashboards display, and nonviability verdicts of the kind that justify tightening a plan menu, can carry their certificates with them wherever verdicts must be re-derivable by third parties. With independent checking, the scaling evidence, and the comparison of Section 5 in place, the tool closes this practical gap within its stated scope. Extensions under way are continuous-state data with certified enclosures, the extension of the weight partition to richer plan menus, and additional certificate families from the obstruction calculus; the interface intentionally leaves room for all three.

CRediT authorship contribution statement

A.A.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of generative AI in the research process

Deepseek AI and Qwen (Alibaba) assisted with code development. The author reviewed and edited the content as needed and takes responsibility for the content.

Acknowledgements

Not applicable.

References

- Abaee, A., 2026a. Aggregate Indices and Transition Safety: A Quantifier-Order Separation Between Scalarized and Coordinate-Wise Feasibility. Manuscript submitted for publication.
- Abaee, A., 2026b. An Obstruction Calculus for Viability under Incomplete Observation. Manuscript submitted for publication.
- Avis, D., 2000. A revised implementation of the reverse search vertex enumeration algorithm, in: Kalai, G., Ziegler, G.M. (Eds.), *Polytopes — Combinatorics and Computation*. Springer, pp. 177–198. https://doi.org/10.1007/978-3-0348-8438-9_9.
- Bruns, W., Ichim, B., 2010. Normaliz: algorithms for affine monoids and rational cones. *Journal of Algebra* 324 (5), 1098–1113. <https://doi.org/10.1016/j.jalgebra.2010.01.031>.
- Caron, S., 2022. pyfme: symbolic polyhedron projection by Fourier–Motzkin elimination using SymPy. Python package, version of 2 December 2022. Available at: <https://github.com/stephane-caron/pyfme>.
- de Moura, L., Bjørner, N., 2008. Z3: an efficient SMT solver, in: Ramakrishnan, C.R., Rehof, J. (Eds.), *Tools and Algorithms for the Construction and Analysis of Systems*, 14th International Conference, TACAS 2008, Budapest, Hungary, March 29–April 6, 2008. Proceedings. Springer, Berlin, Heidelberg, pp. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24.
- Kwiatkowska, M., Norman, G., Parker, D., 2011. PRISM 4.0: verification of probabilistic real-time systems, in: Gopalakrishnan, G., Qadeer, S. (Eds.), *Computer Aided Verification*, 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14–20, 2011. Proceedings. Springer, Berlin, Heidelberg, pp. 585–591. https://doi.org/10.1007/978-3-642-22110-1_47.
- Bakker, M., Post, V., Langevin, C.D., Hughes, J.D., White, J.T., Starn, J.J., Fienen, M.N., 2016. Scripting MODFLOW model development using open source Python and FloPy. *Groundwater* 54 (5), 733–739. <https://doi.org/10.1111/gwat.12413>.
- Coron, L., Thirel, G., Delaigue, O., Perrin, C., Andréassian, V., 2017. The suite of lumped GR hydrological models in an R package. *Environmental Modelling & Software* 94, 166–171. <https://doi.org/10.1016/j.envsoft.2017.05.002>.

- Craig, J.R., Brown, G., Chlumsky, R., Jenkinson, R.W., Jost, G., Lee, K., Mai, J., Serrer, M., Sgro, N., Shafii, M., Snowdon, A.P., Tolson, B.A., 2020. Flexible watershed simulation with the Raven hydrological modelling framework. *Environmental Modelling & Software* 129, 104728. <https://doi.org/10.1016/j.envsoft.2020.104728>.
- Abace, A., 2026. Verification code and figure pipeline for Aggregate Indices and Transition Safety [data set]. figshare. <https://doi.org/10.6084/m9.figshare.33764023>.
- Herman, J., Usher, W., 2017. SALib: An open-source Python library for sensitivity analysis. *Journal of Open Source Software* 2 (9), 97. <https://doi.org/10.21105/joss.00097>.
- Houska, T., Kraft, P., Chamorro-Chavez, A., Breuer, L., 2015. SPOTting model parameters using a ready-made Python package. *PLOS ONE* 10 (12), e0145180. <https://doi.org/10.1371/journal.pone.0145180>.
- Jakeman, A.J., Letcher, R.A., Norton, J.P., 2006. Ten iterative steps in development and evaluation of environmental models. *Environmental Modelling & Software* 21 (5), 602–614. <https://doi.org/10.1016/j.envsoft.2006.01.004>.
- Pianosi, F., Sarrazin, F., Wagener, T., 2015. A Matlab toolbox for global sensitivity analysis. *Environmental Modelling & Software* 70, 80–85. <https://doi.org/10.1016/j.envsoft.2015.04.009>.
- Tomlinson, J.E., Arnott, J.H., Harou, J.J., 2020. A water resource simulator in Python. *Environmental Modelling & Software* 126, 104635. <https://doi.org/10.1016/j.envsoft.2020.104635>.